

RAPPORT

Développement d'une application de recommandation de bouquins pour les étudiants universitaire en appliquant la méthode agile

Master d'excellence en IA

Module: Méthode Agile

Réalisé par :

- Abdelhaq EL AMRAOUI
- Anass BOURASS
- Hamza BELKADI

2025-2026

**MACHINE
LEARNING**

Table de matières

Table de figures	4
Introduction	5
I. Sprint 1 : Analyse et conception	6
1. Les besoins Fonctionnels de l'application	6
2. Conception	7
a. Vue globale sur l'application	7
b. Flowchart de l'application	7
c. Schéma de flux d'une application web : Vercel, FastAPI et Firebase	8
d. Le flux de l'authentification	9
e. Maquettes UI	10
II. Sprint 2 : Développement IA	12
1. Préparation des données	12
2. Approche Technique et Algorithme	12
3. Entraînement du modèle.....	13
4. Tests et Validation.....	13
5. Déploiement et Intégration API	13
III. Sprint 3 : Développement applicatif	14
1. API Backend	14
a. Périmètre et fonctionnalités	14
b. Architecture et persistance	14
c. Sécurité et observabilité	14
2. Intégration du modèle IA	15
d. Serving et performance.....	15
e. Monitoring et gestion des versions	15
f. Intégration côté interface utilisateur	15
IV. Sprint 4 : Tests et validation	15
1. Tests fonctionnels	15
Conclusion.....	17

Table de figures

Figure 1: Sitemap de l 'application	7
Figure 2: Diagramme du flux de l'application	8
Figure 3: Diagramme de flux des service utilisés de l'application	9
Figure 4: Diagramme de séquence de l'authentification	10
Figure 5: Interface UI de gestion de notes	11
Figure 6: Dashboard de l'interface UI	11

Introduction

Ce projet s'inscrit dans le cadre d'un contexte pédagogique visant l'apprentissage et la mise en pratique des méthodes agiles, ainsi que l'utilisation de l'outil Jira pour la gestion et le suivi de projet. Il a pour objectif de concevoir et développer une application web académique intégrant un système de recommandation personnalisée de livres, adapté au profil et au parcours des étudiants.

Le projet a été conduit selon une approche agile, structurée en sprints planifiés et suivis via Jira. Il couvre l'analyse et la conception fonctionnelle et technique de l'application, le développement d'un modèle de recommandation basé sur la vectorisation TF-IDF et la similarité cosinus, ainsi que son intégration au sein d'une API REST. Les dernières phases sont consacrées au développement applicatif, aux tests, à la validation et à la documentation finale.

Ce rapport présente l'ensemble des travaux réalisés, en mettant en évidence les choix méthodologiques et techniques adoptés dans un cadre académique orienté pratiques professionnelles.

I. Sprint 1 : Analyse et conception

1. Les besoins Fonctionnels de l'application

Avant de procéder aux phases de conception et de développement, une analyse des besoins fonctionnels de l'application a été réalisée. Cette étape a permis d'identifier les fonctionnalités essentielles nécessaires au bon fonctionnement du système, en tenant compte des différents profils d'utilisateurs et des objectifs pédagogiques du projet. Les besoins fonctionnels définis ci-après constituent la base de référence pour la conception de l'architecture, le développement des fonctionnalités et la planification des sprints.

1. Authentification et Gestion des Comptes

Permettre aux utilisateurs de créer un compte, se connecter et se déconnecter avec différenciation des rôles (étudiant/administrateur).

2. Gestion des Profils Étudiants

Créer, consulter, modifier et supprimer les profils étudiants avec leurs informations personnelles, domaine d'étude et modules.

3. Gestion des Modules et Domaines d'Étude

Structurer et gérer les domaines d'étude et les modules associés.

4. Gestion des Notes et Résultats Académiques

Saisir, modifier et consulter les notes des étudiants pour chaque module.

5. Système de Recommandation Personnalisée de Livres

Recommander des livres pertinents basés sur le domaine d'étude et les modules de l'étudiant avec un score de pertinence.

6. Recherche et Consultation de Livres

Permettre la recherche de livres par titre, auteur ou sujet avec affichage des informations détaillées.

7. Tableau de Bord Personnalisé

Offrir un dashboard adapté au rôle de chaque utilisateur (étudiant ou administrateur).

8. Rapports et Analytiques

Fournir des statistiques et graphiques sur la performance des étudiants et les tendances académiques.

9. Sécurité et Contrôle d'Accès

Sécuriser les données et mettre en place un contrôle d'accès basé sur les rôles.

10. Interface Utilisateur Responsive

Offrir une interface intuitive et accessible sur tous les appareils.

2. Conception

a. Vue globale sur l'application

Nous illustrons la vue globale de l'application et des pages avec le diagramme suivant :

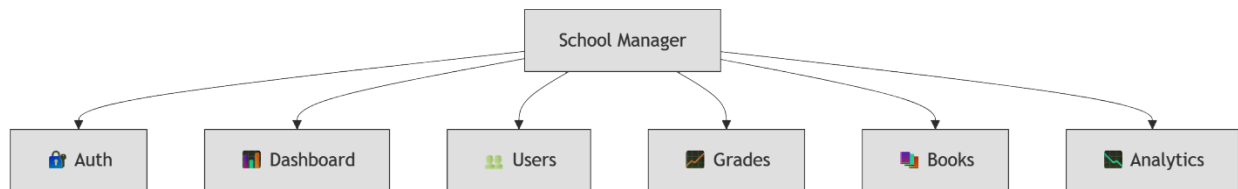


Figure 1: Sitemap de l'application

b. Flowchart de l'application

Ce diagramme illustre le flux de navigation principal de l'application en fonction de l'état d'authentification et du rôle de l'utilisateur. Il décrit le parcours depuis l'accès initial à la plateforme jusqu'aux fonctionnalités disponibles, en distinguant les actions accessibles aux étudiants et aux administrateurs. Ce schéma permet de visualiser le contrôle d'accès, la gestion des rôles et l'orientation de l'utilisateur vers les services appropriés, assurant ainsi une utilisation sécurisée et structurée de l'application.

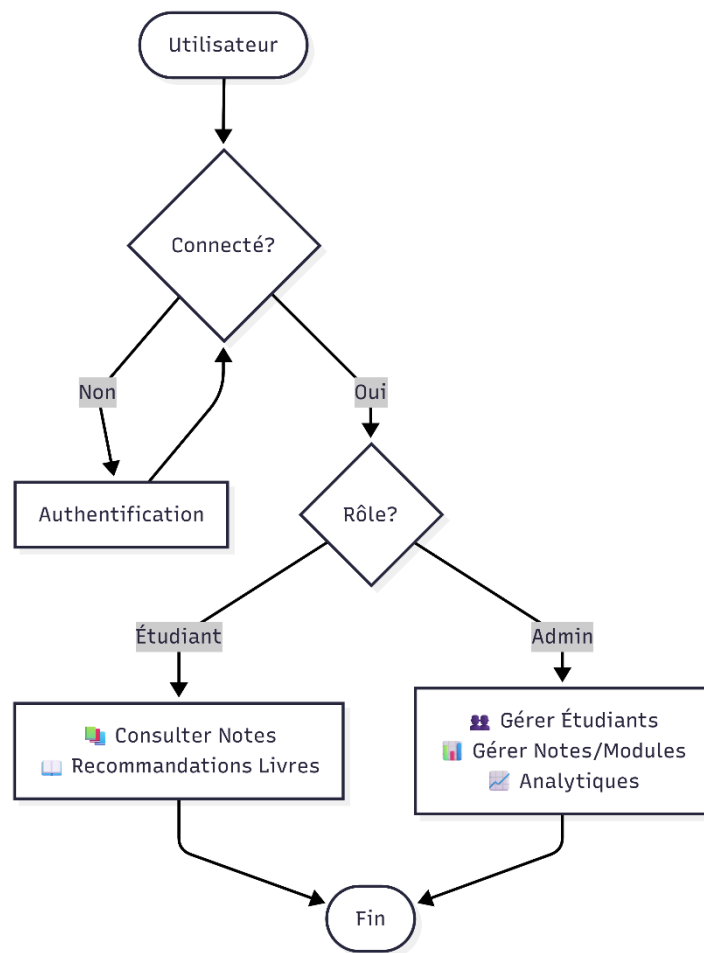


Figure 2: Diagramme du flux de l'application

c. Schéma de flux d'une application web : Vercel, FastAPI et Firebase

Nous présentons l'architecture globale de l'application web, mettant en évidence les interactions entre le frontend déployé sur Vercel, le backend FastAPI, Firebase Authentication et la base de données Firestore. Il illustre le flux d'authentification basé sur

les tokens JWT, la sécurisation des échanges et la circulation des données entre les différentes couches du système.

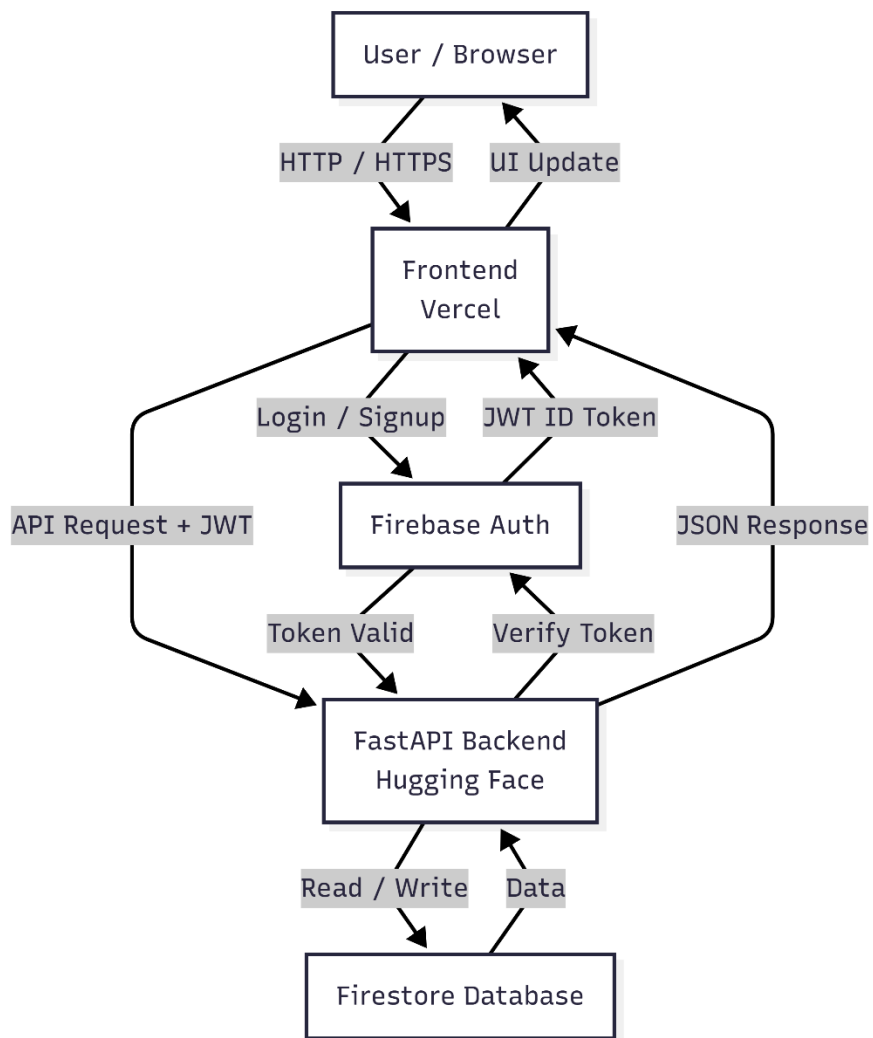


Figure 3: Diagramme de flux des service utilisés de l'application

d. Le flux de l'authentification

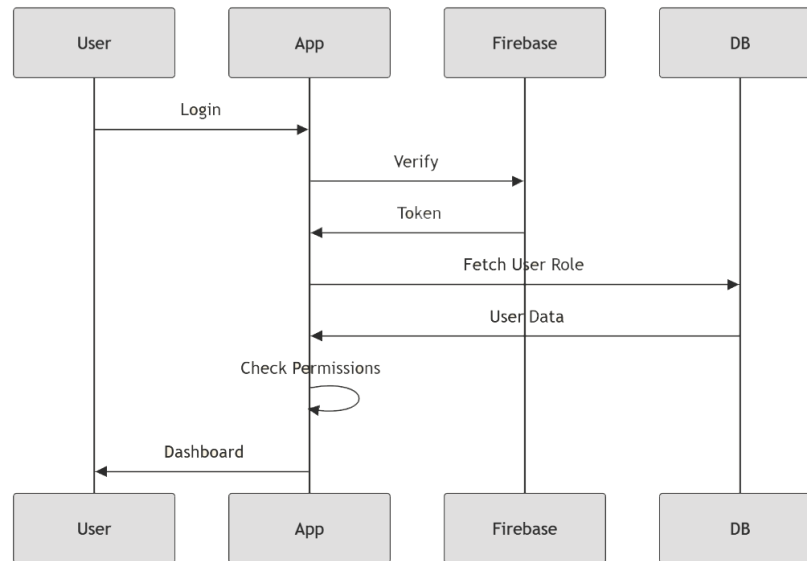


Figure 4: Diagramme de séquence de l'authentification

e. Maquettes UI

Pour de la conception des maquettes UI, une approche pragmatique a été adoptée en procédant directement à la génération de l'interface utilisateur à l'aide d'outils basés sur l'intelligence artificielle. Cette méthode a permis d'obtenir rapidement des interfaces fonctionnelles et cohérentes, tout en accélérant la phase de prototypage et en facilitant les itérations sur l'ergonomie et l'expérience utilisateur.

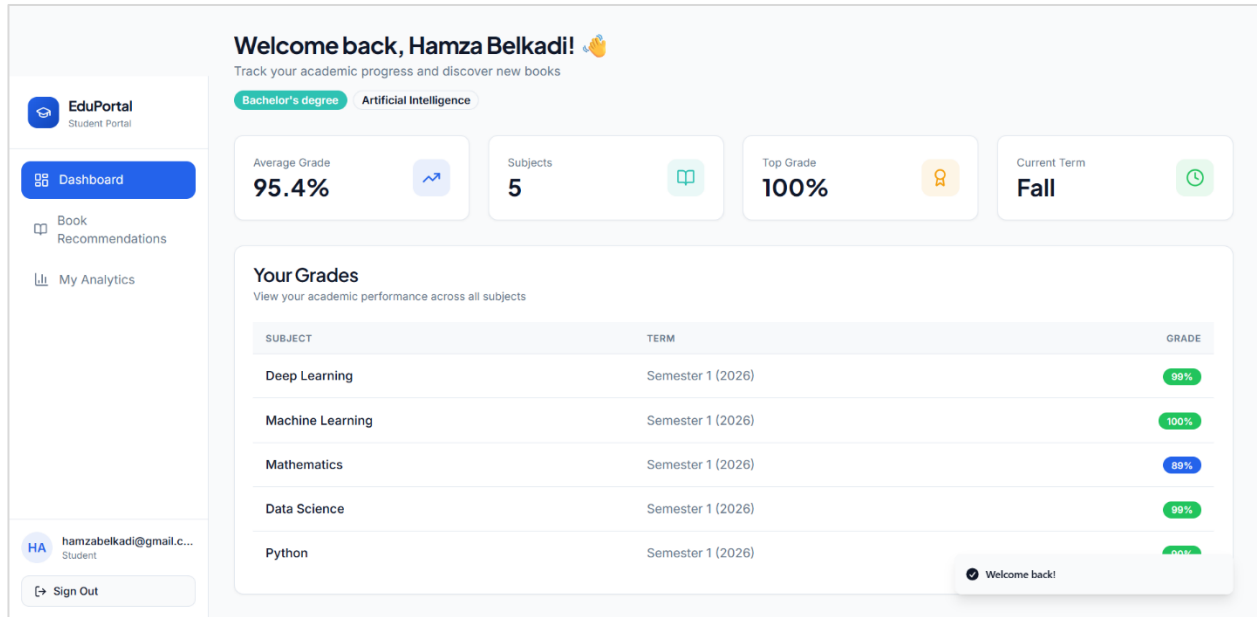


Figure 6: Dashboard de l'interface UI

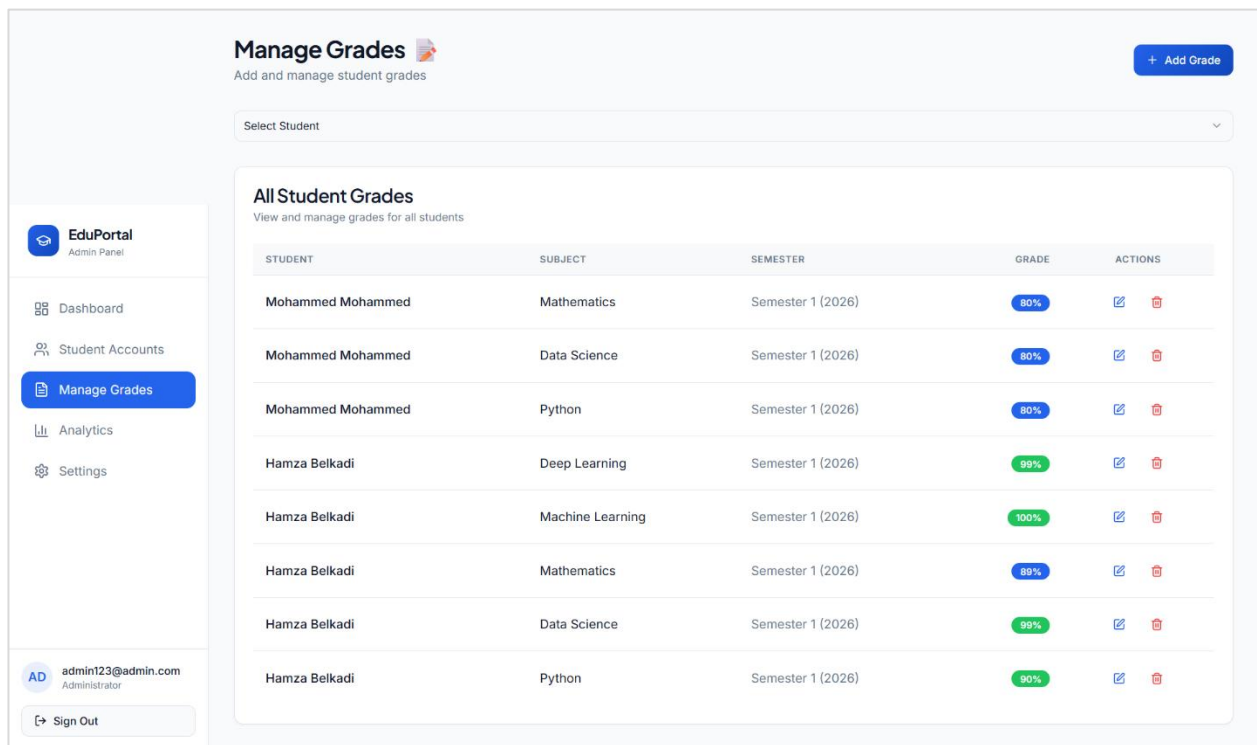


Figure 5: Interface UI de gestion de notes

II. Sprint 2 : Développement IA

1. Préparation des données

Dans le cadre du développement du système de recommandation académique, deux sources de données principales ont été utilisées :

Base de données étudiants : Un fichier Students.csv contenant les profils académiques des étudiants avec leurs domaines d'études et modules suivis (module_1 à module_5). Pour optimiser les performances lors de la phase de développement, un échantillon de 1000 étudiants a été sélectionné.

Base de données livres : Un fichier Books.csv regroupant une collection de ressources pédagogiques avec leurs identifiants, titres et résumés. Un échantillon aléatoire de 1000 livres a été extrait pour maintenir un équilibre entre la diversité des recommandations et l'efficacité du système.

Le processus de nettoyage des données a consisté à fusionner les caractéristiques pertinentes de chaque entité. Pour les étudiants, les colonnes domaine et modules ont été concaténées en une seule chaîne de caractères représentative de leur profil académique. De même, pour les livres, les titres et résumés ont été combinés pour créer une description textuelle complète de chaque ressource.

2. Approche Technique et Algorithme

L'approche retenue repose sur le calcul de similarité cosinus entre les profils étudiants et les caractéristiques des livres. Cette méthode de filtrage basé sur le contenu permet d'identifier les ressources les plus pertinentes en fonction du parcours académique de chaque étudiant.

Vectorisation TF-IDF : La technique TF-IDF (Term Frequency-Inverse Document Frequency) a été appliquée pour transformer les textes en vecteurs numériques. Cette méthode attribue un poids plus important aux termes spécifiques et rares, tout en réduisant l'influence des mots courants. Le vectoriseur a été entraîné sur l'ensemble du corpus de livres afin de créer un espace de référence stable.

Calcul de similarité : Pour chaque requête d'un étudiant, le système transforme son profil académique en vecteur à l'aide du même vectoriseur TF-IDF, puis calcule la similarité cosinus entre ce vecteur et tous les vecteurs de livres. Les 10 livres présentant les scores de similarité les plus élevés sont alors retournés comme recommandations.

3. Entraînement du modèle

Nous avons entraîné le modèle en créant la matrice TF-IDF des 1000 livres de référence. Pour garantir la persistance du système et faciliter son déploiement, trois composants essentiels ont été sérialisés dans un fichier `recommender_model.pkl` :

- Le vectoriseur TF-IDF entraîné
- La matrice TF-IDF des livres
- Les métadonnées des livres (identifiants et titres)

Cette approche permet de charger rapidement le modèle sans nécessiter un nouvel entraînement à chaque utilisation.

4. Tests et Validation

Les tests fonctionnels ont été réalisés pour valider la pertinence du système. Un profil étudiant fictif dans le domaine "Religion and Church" avec des modules tels que "Worship", "Pastors" et "Christian Education" a été soumis au système.

Les résultats ont démontré que le modèle identifie correctement les ressources pertinentes en fonction du profil académique. Les scores de similarité cosinus permettent de classer les recommandations par ordre de pertinence, les livres les plus similaires au profil de l'étudiant obtenant les scores les plus élevés.

5. Déploiement et Intégration API

Nous avons transformé le modèle en API REST pour permettre son intégration dans l'interface web. L'API reçoit en entrée les données du profil étudiant (domaine et modules) saisies via l'interface utilisateur, les transforme en vecteur TF-IDF, calcule les similarités avec la base de livres, et retourne les 10 meilleures recommandations sous forme de dictionnaire contenant les identifiants et titres des livres suggérés.

Cette architecture modulaire garantit la séparation entre la couche intelligence artificielle et la couche présentation, facilitant ainsi la maintenance et les évolutions futures du système.

III. Sprint 3 : Développement applicatif

1. API Backend

a. Périmètre et fonctionnalités

- Service de recommandation de livres exposant un point de santé et un Endpoint de recommandations basées sur les préférences (domaine, modules, limite).
- Intégration dans l'interface pour consommer et présenter les recommandations de façon claire et contextualisée.
- Validation des entrées (domaine, modules non vides, borne supérieure sur le nombre de résultats).

b. Architecture et persistance

- Service REST s'appuyant sur TF-IDF et similarité cosinus, avec chargement du modèle (vecteur TF-IDF, matrice et métadonnées) en mémoire au démarrage pour minimiser la latence.
- Stockage du modèle sous forme d'artefact versionné (chemin local ou URL distante), avec identification par version, horodatage et métriques d'entraînement.
- Procédure documentée de rafraîchissement et de rollback du modèle pour assurer la traçabilité.

c. Sécurité et observabilité

- Restriction des origines autorisées (CORS) et mise en place d'une authentification pour un accès public.
- Validation renforcée des payloads pour prévenir les requêtes malformées.
- Logging structuré (requêtes, durée, taille des réponses, erreurs) et métriques opérationnelles (compteur de requêtes, taux d'erreur, disponibilité du modèle).

2. Intégration du modèle IA

d. Serving et performance

- Chargement unique du modèle en mémoire et traitement synchrone des requêtes.
- Limitation du nombre de recommandations par requête ; suivi du temps moyen de réponse et de l’empreinte mémoire.
- Anticipation d’optimisations (indexation ou stratégies approximatives) si la volumétrie augmente.

e. Monitoring et gestion des versions

- Exposition de métriques clés (latence, succès/erreur, statut et version du modèle).
- Versionnage explicite du modèle (ID, date, métriques d’entraînement) et journalisation des mises à jour/rollback.
- Enrichissement du point de santé pour refléter la source et la version du modèle chargé.

f. Intégration côté interface utilisateur

- Consommation directe du service de recommandations et rendu structuré des résultats (classement, score, prix/notes si disponibles).
- Gestion d’erreurs explicites lorsque le service est indisponible ou que le modèle n’est pas chargé, afin de préserver l’expérience utilisateur.

IV. Sprint 4 : Tests et validation

1. Tests fonctionnels

Dans le cadre du Sprint 4, nous nous concentrons sur les tests fonctionnels afin de nous assurer que toutes les fonctionnalités que nous avons développées répondent aux exigences spécifiées et fonctionnent correctement. Ces tests ont pour objectif de valider, dans des conditions réelles, que chaque fonctionnalité du système fonctionne comme prévu. Nous vérifions ainsi que les critères d'acceptation définis dans les sprints précédents sont bien respectés. On procède à ces tests en collaboration avec l'équipe QA, en testant les interactions entre les différents modules, en s'assurant de l'intégrité des données et en vérifiant la conformité des résultats attendus. L'objectif est de détecter rapidement les

erreurs et incohérences, afin de les corriger avant la phase de validation finale. Cela nous permet de garantir la fiabilité et la qualité du produit avant sa mise en production.

Les flux critiques ont été validés :

- Authentification complète (création compte → connexion → accès pages protégées)
- Gestion des données (CRUD Firebase, synchronisation temps réel)
- Navigation sécurisée avec vérification des permissions
- Intégration API livres avec gestion des erreurs

Conclusion

Ce projet a permis de mettre en œuvre, dans un cadre académique, une application web complète intégrant à la fois des fonctionnalités de gestion académique et un système de recommandation basé sur l'intelligence artificielle. Réalisé selon une approche agile et suivie à l'aide de l'outil Jira, il a offert une vision concrète des différentes phases d'un projet logiciel, de l'analyse des besoins jusqu'aux tests et à la documentation finale.

Le développement du système de recommandation, fondé sur la vectorisation TF-IDF et le calcul de similarité cosinus, a démontré la pertinence des techniques de filtrage par le contenu pour proposer des ressources pédagogiques adaptées aux profils académiques des étudiants. L'intégration de ce modèle via une API REST, au sein d'une architecture modulaire combinant frontend, backend et services cloud, garantit la maintenabilité et l'évolutivité de la solution.

Au-delà des aspects techniques, ce projet a renforcé la compréhension des pratiques professionnelles liées aux méthodes agiles, à la gestion des sprints et au travail collaboratif. Il constitue ainsi une base solide pouvant être enrichie par des évolutions futures, telles que l'amélioration du modèle de recommandation, l'intégration de nouvelles sources de données ou l'ajout de fonctionnalités analytiques avancées, en vue d'une exploitation à plus grande échelle.